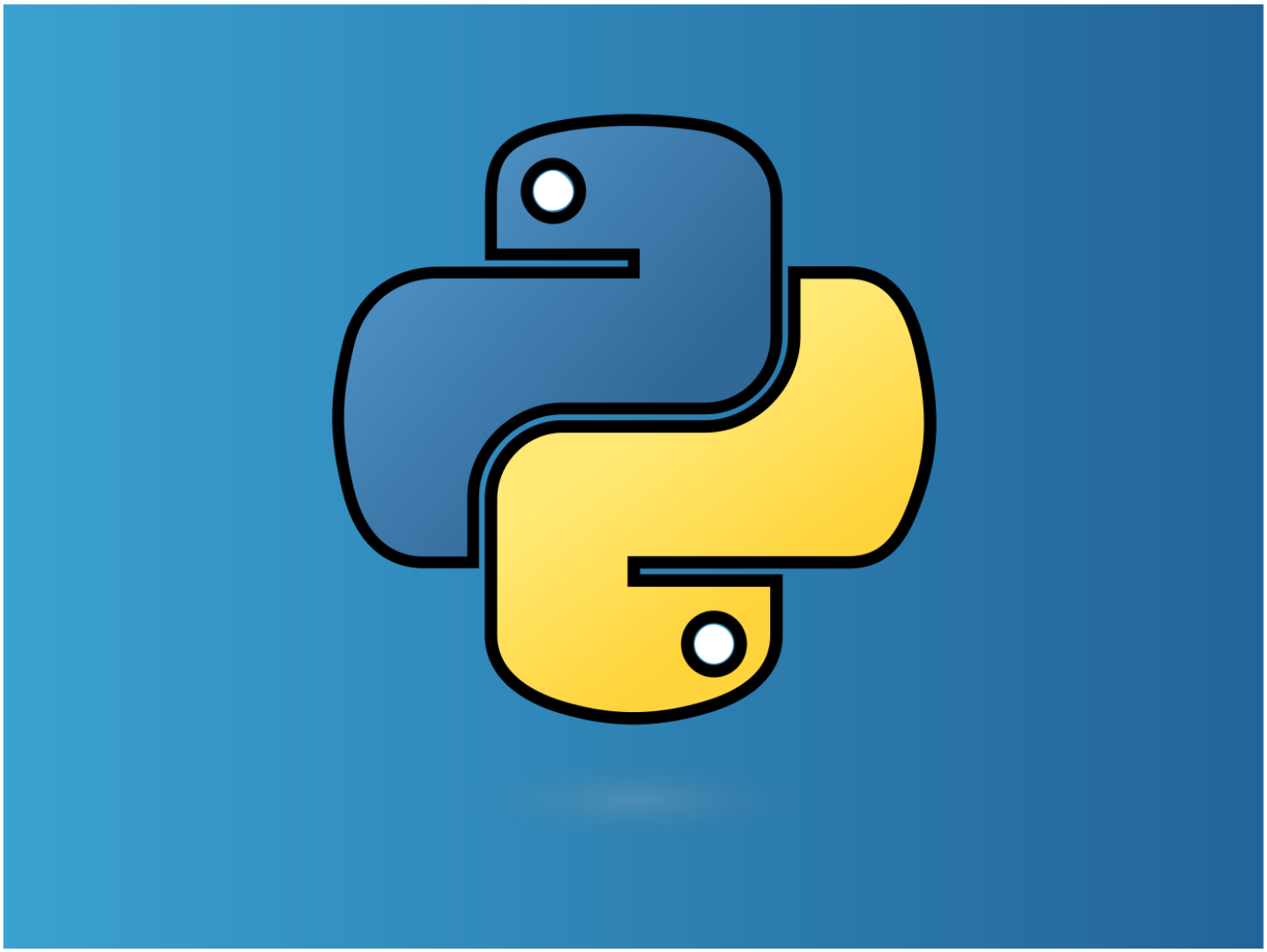
Chapter 1. Python data types and structures (A Python brief summary)



Data types

Python has many built-in data structures: tuples, lists, dictionaries, sets, etc... These can be used if necessary to build custom data objects that fit better to the programmer's needs. Let's make a brief recapitulation of Python properties.

Python offers some built-in basic data types, such as:

- None
- Integers
- Floats
- Booleans
- Strings
- Tuples
- Lists
- Dictionaries
- Sets

There are more built-in data types, but probably the most common ones are these. They all are also objects. We use these basic data objects usually through variables, like:

```
integer_var = 3
string_var = "Hello"
```

Variables

Remember how variables work in Python. Sometimes, when explaining programming to learners, variables are compared to named boxes that can contain some value. And that can be good to understand the concept, but that is not how Python stores data in variables. Python stores the data somewhere in memory and a variable is actually a **pointer to that memory location**. Python hides that complexity from us and most of the time we can just assume that the variable *contains* the value itself, but we have to be aware that this is not true as sometimes the way Python handles variables has real consequences for our programs. So, if you do something like:

```
a = 5
b = a
```

Then `a` and `b` are both pointers or labels pointing to the same object in Memory (the integer number 5). With immutable objects (Integers, Floats, Boolean, Sets, ...) this does not affect very much, but with mutable objects, like Python lists or dictionaries, it can have unexpected effects. For example:

```
a = [1, 2, 3]
b = a
b[0] = 5     # This also modifies "a"
print(a)
# [5, 2, 3]
```

So be very careful when modifying variables that are, somehow, "copies" of some other objects. This is especially true with function parameters. Inside functions you can use parameters, which are copies of some variables outside the function, but the objects themselves are the same. So, if you modify an object inside a function, it's also modified for the variables that refer it outside the function. Notice that a simple variable assignment does not modify the underlying object but just the variable "pointer". For example:

```
a = 5
b = a       # Now 'b' also points to the number 5
b = 6       # Now 'b' points to a memory place with the number 6
print(a)    # But the original 'a' variable still points to the number 5
```

One of the most useful and used types in Python are **lists**. Python lists are a set of ordered objects that can be accessed by their index. Python lists are widely used, and Python offers a wide variety of tools and methods for working with lists.

Tuples work mainly the same way as lists, but they are immutable (can't be modified after creation). And **sets** are also a collection of objects but they can't be accessed by an index. The main distinctive property of sets is that it can't contain duplicates.

Lists, tuples, sets and strings are also sequences. That is, they are sets of python objects ordered in a certain way and indexed by positive integers (with the exceptions that strings can only contain characters and sets are not indexable).

```
list_var  = ['a', 'b', 'c', 10, 20, True]   # It can contain different types
tuple_var = ('a', 'b', 'c', 10, 20, True)   # It can contain different types, but tuple can't
be modified after creation
set_var   = {'a', 'b', 'c', 10, 20, True}   # It can contain different types but not duplicates
string_var = "abc"                          # It can contain only characters

# Indexing:
list_var[0]     # 'a'
tuple_var[0]    # 'a'
string_var[0]   # 'a'
# Set's elements can't be accessed with indexing
```

In Python sequences are also "iterables", that is, that you can traverse through all the elements of the sequence in order.

```
string_var = 'abc'
for character in string_var:
    print(character)
# a
# b
# c

# Note: Exactly the same code can be applied to lists, tuples and sets.
```

Some other data types can also be iterables, like **dictionaries**, even if they are not sequences *per se*. But, in the case of dictionaries, you can iterate, for example through the keys (the default action) or through the values:

```python
dict_var = {'name': 'Mika', 'age': 35, 'is_member': True}
for elem in dict_var:       # Also dict_var.keys()
    print(elem)
# 'name'
# 'age'
# 'is_member'


for elem in dict_var.values():
    print(elem)
# 'Mika'
# 35
# True


# Remember that you can also iterate through both:
for key, value in dict_var.items():
    print(key, value)
# 'name' 'Mika'
# 'age' 35
# 'is_member' True
```

Remember, when talking about variables, that they have scopes. That is, they have certain boundaries wherein the variable is defined and functional. You can read more about that, if necessary, in the Python documentation.

Flow control

Python programs consist of one or more statements. In principle, the Python interpreter executes each statement in order until there are no more statements. But we have the power of controlling what statements are executed and how many times through the use of flow control structures. The two main types of control structures are conditional statements and loops.

For conditional statement, Python offers the `if .. else` structure. This basic control structure is present in almost every programming language somehow. In Python it is used like this:

```
if some_var == 1:
    # execute something
elif some_var == 2:
    # execute some other thing
else:
    # execute something else
```

Being the `elif` and `else` parts just optional. Notice that the conditions are defined as conditional expressions that return True or False (or some value that can be understood as True or False. These are known as truthy and falsy values respectively). In this example the conditions use the `==` comparison operator that checks that the left side of the expression and the right side of the expression are equals.

For looping Python offers `for` and `while`. The `for` structure is used specially when it's known beforehand how many times the loop has to run (usually in connection with `range`) or when traversing an iterable (as we saw earlier)

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4

for v in ['a', 'b', 'c']:
    print(v)
# a
# b
# c
```

The `while` loop works similarly, but we may not know in advance how many times the loop is going to run:

```
a = 0
while a < 100:
    b = int(input('Say a number: '))
    a += b
```

Remember that you can also use `continue` and `break` statements inside the loops to continue onto the next loop or to break out of the loop respectively.

Operators

There are a lot of different kinds of operators in Python. We will not go thoroughly here, but some common operators are:

- Arithmetic: To perform arithmetic calculations. Ex: `+`, `-`, `*`, `/`, ...
- Assignment: To assign values to variables. Ex: `=`, `+=`, `-=`, ...
- Comparison: To compare values and return a True/False value. Ex: `==`, `!=`, `<`, `>`, `<=`, ...
- Logical: To perform a Boolean calculation between two or more values and return a True/False value. Ex: `and`, `or`, `not`
- Identity: To check if two objects are actually the same object (and not two objects with same value). Ex: `is`, `is not`
- Membership: To check if a value is inside a sequence. Ex: `in`, `not in`
- Bitwise: To perform bit -level operations. Ex: `&`, `|`, `^`, `~`, `<<`, `>>`

You can read more about basic types and operators in Python documentation (also this article is nice.)

Functions

Python uses functions as well as many other programming languages to build blocks of code that can be reused and help to make the code more readable and understandable.

```
def my_function(parameter):
    # Do something inside the function
    return f'You passed this parameter to the function: {parameter}'
```

For more information on functions, you can read the Python documentation.

Functions are a basic tool in any programming task and coders should know how to use them (including parameter handling) and benefit from them.

Objects and classes

Another key component of many programs are objects. Object oriented programming (OOP) is part of many modern programming languages and coders should know about its concepts even if they don't use them. In OOP there are the concepts of classes and objects. A class is (in theory) an abstract way of defining some properties and functionalities of some "thing" or object.

The typical example is to say that person is a class and that a concrete person is an object. As a class, person has properties like height, weight, age, name, etc... and an object is an "instantiation" of that class, i.e., a class instance that can have different values for those properties than other class instance.

Class functions are actually called "methods" and class properties are... properties, but they are just class/object variables. A class is defined like this:

```
class MyClass():
    def __init():
        self.my_property = 1

    def increase(self):
        self.my_property += 1

    def decrease(self):
        self.my_property -= 1
```

As probably the student already know, properties and methods can be accessed with the dot notation. For example:

```
my_object = MyClass()
print(my_object.my_property)    # 1
my_object.increase()
print(my_object.my_property)    # 2
```

This is another topic that can be quite broad and can't be explained here with enough details. Therefore, students are encouraged to learn this topic well before taking this course. Among other subjects, the student must know about inheritance, the so-called magic methods, private variables and class vs object properties/variables

Miscellaneous

Here is also a list of recommended topics that students should know at some degree to successfully complete this course:

- Comprehensions
- Main Python's data types methods:
  - String methods: `join`, `split`, `lower`, `upper`, `replace`, ...
  - List methods: `append`, `insert`, `pop`, `remove`, `reverse`, `sort`, ...

    - And so on ...
- Generators
- Lambda functions
- Exception handling
- regular expressions

Python has a "batteries included" philosophy. That means that by default Python offers a wide range of modules that can help programmers do their work instead of reinventing the wheel or search for third party extensions on internet. Python coders should know about the most used modules that come included with Python. For example: `json`, `random`, `csv`, `os`, `sys`, `re`, `math`, `datetime`, ...

That said, not always these modules or libraries fill the needs of the programmer. So, it's good to have the skills to make own custom modules or data objects if necessary.

Exercises

Please complete the exercises you will find in chapter 1 of the Moodle environment.